Nifty Volatility Forecaster

Project structure, stack, configuration and code skeletons

A NumPy-from-scratch system that forecasts next-day Nifty 50 realized volatility with six models: Naive, HAR-RV, GARCH(1,1), MLP, LSTM and a time-series Transformer, with news sentiment as an ablation. No ML libraries. No pretrained models.

Prepared from: Concepts From Zero (21 Oct 2026) and Build Roadmap (21 Oct 2026).
Purpose: feed to GitHub Copilot and make the initial commit.
Companion file: PROJECT_BLUEPRINT.md (same content, Markdown).

Contents

Contents 2

1. How to use this document 4

2. Project analysis 4

2.1 What we are building . . . 4

2.2 Model ladder . . . 4

2.3 Data, split and evaluation . . . 5

2.4 Datasets . . . 5

2.5 Decisions and corrections made while consolidating the two documents . . . 5

3. Hard constraints (no pretrained models, no ML libraries) 6

4. Technology stack and configuration 6

5. Repository structure 7

6. Data contracts between modules 8

6.1 daily_features.csv (one row per trading day) . . . 8

6.2 Forecast CSV (forecasts/.csv) . . . 8

6.3 Alignment rules (the leakage checklist) . . . 8

7. Code skeletons and explanations 8

7.1 Configuration and I/O . . . 9

7.2 Data pipeline (Phase 0-1) . . . 10

7.3 NumPy neural-network core (Phase 3) . . . 13

7.4 Classical baselines (Phase 2) . . . 17

7.5 Shared model plumbing and the MLP (Phase 3) . . . 18

7.6 LSTM (Phase 4) . . . 19

7.7 Time-series Transformer (Phase 6) . . . 20

7.8 News sentiment from scratch (Phase 5) . . . 22

7.9 Evaluation (Phase 7) . . . 24

7.10 Entry-point scripts . . . 25

8. GitHub Copilot guidance 27

8.1 .github/copilot-instructions.md . . . 27

8.2 Per-module prompts for Copilot Chat . . . 28

9. Tests, CI and phase checklist 29

9.1 Tests (they define 'done' for the skeleton) . . . 29

9.2 Phase checklist (what each phase must produce) . . . 31

9.3 Risks and fallbacks (condensed from the roadmap) . . . 31

9.4 Pitfalls checklist for code review . . . 31

10. Initial commit procedure 32

10.1 Commands . . . 32

10.2 data/raw/SOURCES.md template . . . 32

10.3 README.md skeleton 32
10.4 Suggested first-week order of work 32

1. How to use this document

This document is the **project blueprint** for the Nifty Volatility Forecaster. It turns the two source documents (Concepts From Zero, Build Roadmap) into one engineering spec: what we build, which stack and rules apply, the exact repository layout, the data contracts between modules, and working code skeletons with an explanation next to every piece. It is written so that GitHub Copilot (or any code assistant) can read it and generate consistent code, and so that a human reviewer can follow every design decision.

Recommended workflow for the initial commit

- Create an empty GitHub repository named `nifty-vol` and clone it.
- Copy this PDF (and the companion Markdown file) into `docs/` in the repo.
- Create the folders and empty `__init__.py` files from section 5, then paste the files from section 7 and 9. Section 10 lists the exact git commands.
- Paste the Copilot instruction file (section 8.1) into `.github/copilot-instructions.md`; Copilot then reads the project rules automatically on every request.
- In Copilot Chat, attach `docs/PROJECT_BLUEPRINT.md` (or this PDF) and use the per-module prompts in section 8.2.
- Run `python -m tests.run_tests`. The skeleton's gradient checks should already pass. Commit.

What 'skeleton' means here: foundational pieces (layers, optimiser, gradient checker, attention, LSTM cell, metrics, DM and Kupiec tests) are written in full because every later model depends on them being exactly right. Dataset-specific parts (column names of the Kaggle/NSE files) are marked `TODO` *because they cannot be known without the files in hand.*

Verification status. Every code block in sections 4, 7, 8, 9 and 10 was extracted into a fresh repository and executed before this document was finalised: all 10 tests pass (gradient checks for Dense, ReLU, GELU, LayerNorm, Softmax, both losses, the tiny LSTM, multi-head attention, the encoder block and the full Transformer; metric, DM and Kupiec tests; the no-leakage test; the banned-import scan), and the whole chain features -> baselines -> MLP/LSTM/Transformer -> evaluation tables ran end to end on synthetic data. Reported sizes: MLP 6,657 parameters, LSTM 6,049, Transformer 9,761 (14 inputs) and 9,857 (17 inputs). The real datasets were not available, so the `TODO` loaders have not been run on them.

2. Project analysis

2.1 What we are building

A system that forecasts **how much the Nifty 50 will move tomorrow (volatility), not which way**. The target is log realized volatility of day t+1, computed from 5-minute returns. Six models of increasing complexity are compared on identical, unseen days with two error measures (MSE on log RV and QLIKE), a Diebold-Mariano significance test, and a Value-at-Risk backtest (Kupiec test). The most advanced model is a time-series Transformer written from scratch in NumPy; news sentiment is one extra input group, tested by ablation.

Why volatility and not price direction: next-day direction of a large index is close to a coin flip, while volatility clusters (calm follows calm, storm follows storm) and so is genuinely forecastable. News shocks raise volatility regardless of direction, so sentiment has a fair chance of helping.

2.2 Model ladder

Rung	Model	Input	Fitted by
0	Naive	today's log RV	no fitting
1	HAR-RV (+ HAR-X)	log RV averaged over 1, 5, 22 days (+VIX, neg. return, gap, sentiment)	OLS (np.linalg.solve)
2	GARCH(1,1)	open-to-close daily returns (in percent)	Gaussian MLE, numerical gradient + Adam
3	MLP	last 5 days x 14 features, flattened (70)	Adam, MSE
4	LSTM	22 days x 14 features, sequence	Adam, MSE, BPTT, grad clipping
5	Transformer (x2)	22 days x 14 (no sentiment) or x 17 (with sentiment)	Adam, MSE, 5 seeds each

2.3 Data, split and evaluation

Item	Specification
Window	Jan 2015 to Jun 2023 (minute data and headline archive overlap), about 2,100 trading days
Bars	1-minute index data resampled to 75 five-minute bars (09:15 to 15:30 IST); drop days with < 60 bars
Train / Val / Test	Jan 2015-Dec 2019 (~1,230) / 2020 (~250, includes COVID) / Jan 2021-Jun 2023 (~620, touched once)
Walk-forward	Refit at each quarter start from Jan 2021, forecast that quarter, slide forward
Errors	MSE on log RV; QLIKE = RV/h - log(RV/h) - 1 (computed from exp of log forecasts)
Significance	Diebold-Mariano with Newey-West variance (about 5 lags), |DM| > 1.96 at 5%
Risk	99% VaR = 2.33 x forecast daily volatility; count breaches; Kupiec LR vs 3.84
Ablations	Transformer +/- sentiment, +/- VIX; LSTM vs Transformer; HAR vs HAR-X (5 seeds for NNs)

2.4 Datasets

Dataset	Use	Lands in
Nifty 50 1-minute OHLC 2015-2024 (Kaggle, no volume)	RV, returns, range	data/raw/
India VIX daily history (NSE)	forward-looking volatility feature	data/raw/
Nifty 50 daily shares traded + turnover (NSE)	activity feature	data/raw/
NSE options expiry calendar (circulars)	expiry-day flag (weekday changed over the years)	data/raw/
India News Headlines 2001-Jun 2023 (Times of India, ~3.9M)	daily news sentiment (business only)	data/raw/
Financial PhraseBank (>=66% agreement)	train the sentiment classifier	data/raw/
Labelled Economic Times headlines (400)	check classifier on Indian news	data/raw/

2.5 Decisions and corrections made while consolidating the two documents

The two source documents are mostly consistent. These are the places where they were ambiguous or slightly inconsistent, and the single decision this blueprint takes (Copilot should follow these):

- **14 features, not 15.** The concepts table lists 10 numeric features plus a 5-column one-hot day of week (15), yet both documents say 14 inputs (17 with sentiment) and size the models from that. A 5-column one-hot plus an intercept is also collinear. Decision: **4 day-of-week dummies (Mon-Thu, Friday is the reference)**, giving exactly 14 base + 3 sentiment = 17.
- **Units.** 'log RV' always means the natural log of realized **variance** (sum of squared 5-minute returns, decimal returns). Volatility = exp(log_rv / 2). One helper `log_rv_to_vol` is used everywhere (VaR bug source).
- **RV window equals open-to-close.** The first 5-minute return of each day is measured from the day's open price, so the squared returns sum exactly over the same window as the open-to-close return that GARCH and the VaR backtest use. No return ever crosses the overnight gap.
- **Forecast CSV has two dates.** `date` = day t when the forecast is issued, `target_date` = the day being forecast. Splits are assigned by `target_date`, so no training target ever lies in the validation period.
- **Target scaling.** log RV is around -9 while a freshly initialised network outputs about 0. Neural models train on the target standardised with train-period mean and std, and predictions are converted back before saving. HAR, GARCH and Naive work on raw log RV.
- **Parkinson feature is logged** (`log_parkinson`) for scale; a tiny epsilon avoids log(0).
- **Layer parameter API.** The roadmap sketch returns `(W, 'dW')` strings; this blueprint instead returns `(value_array, grad_array)` pairs updated in place, which the optimiser, clipper and gradient checker use uniformly. Gradients **accumulate** (+=) after `zero_grad()`, required for the LSTM's shared weights.
- **Sentiment columns are NaN until Phase 5.** Models that do not use sentiment select `BASE_FEATURES` only.
- **Small repo additions** to the roadmap layout: `src/config.py`, `src/io.py`, `src/data/load_aux.py`, `src/nn/functional.py`, `src/nn/trainer.py`, `src/models/common.py`, `scripts/`, `.github/`, `docs/`.

- **pandas / np.linalg are allowed only after written approval** from the professor (Phase 0). A fallback using the `csv` module is in the roadmap risk table. `pytest` is optional; tests also run via plain Python.

3. Hard constraints (no pretrained models, no ML libraries)

Allowed	Not allowed (anything that fits or provides a model for you)
Python 3.10+, NumPy, pandas (CSV loading and date handling), matplotlib (plots), Python standard library (math, re, csv, json, argparse, pathlib, dataclasses)	scikit-learn, PyTorch, TensorFlow, Keras, JAX, statsmodels, arch, NLTK, spaCy, scipy.optimize/stats, HuggingFace transformers, gensim, xgboost, lightgbm, any pretrained embedding or checkpoint

- The Transformer, LSTM, MLP, GARCH fitting, HAR (OLS), TF-IDF, logistic regression, Naive Bayes, Adam, Diebold-Mariano and Kupiec tests are all implemented in this repo with NumPy.
- **Nothing is downloaded as a model.** The only external artefacts are CSV datasets listed in section 2.4. All weights are initialised randomly from a seeded `np.random.default_rng` and trained in this repo.
- The p-values use `math.erfc` (standard library) instead of scipy.
- The rule is enforced by code: `tests/test_no_banned_imports.py` (section 9.1) scans every `.py` file with the `ast` module and fails if a banned library is imported, or if the words `from_pretrained` / `pretrained` appear in code. The CI workflow runs it on every push.

4. Technology stack and configuration

Layer	Choice	Reason
Language	Python 3.10, 3.11 or 3.12	Type hints, dataclasses, wide NumPy support
Numerics	numpy >= 1.26	All model maths; float64 everywhere (gradient checks need it)
Data	pandas >= 2.1	Reading CSVs, date alignment, resampling only; no modelling
Plots	matplotlib >= 3.8	Report figures
Tests	plain asserts + `tests/run_tests.py` (pytest optional)	Keeps dependency list at three libraries
Demo	static HTML + JS charting lib from a CDN, reads exported JSON	No model runs in the browser
CI	GitHub Actions, Python 3.11	Runs tests, gradient checks and banned-import scan
Hardware	CPU only	Models have ~6k to ~10k parameters; a full NN run takes minutes
Reproducibility	`np.random.default_rng(seed)`, seeds 0-4	5 seeds for every neural model

requirements.txt

```
numpy>=1.26,<3
pandas>=2.1,<3
matplotlib>=3.8,<4
```

requirements-dev.txt

```
-r requirements.txt
pytest>=8.0        # optional; tests also run with: python -m tests.run_tests
```

`requirements.txt` deliberately lists only the three allowed libraries so that a reviewer can see the no-ML-library rule at a glance. Install with `python -m venv .venv`, activate it, then `pip install -r requirements.txt`.

.gitignore

```
.venv/
__pycache__/
*.pyc
.ipynb_checkpoints/
.DS_Store
.vscode/
# large raw datasets are not committed; record their source in data/raw/SOURCES.md
data/raw/*
!data/raw/SOURCES.md
!data/raw/.gitkeep
# keep small processed files and forecasts tracked so results are reproducible
```

Makefile

```
PY ?= python

features:   ; $(PY) -m scripts.build_features
sentiment:  ; $(PY) -m scripts.build_sentiment
baselines:  ; $(PY) -m scripts.run_baselines
nn:         ; for m in mlp lstm transformer_nosent transformer_sent; do $(PY) -m scripts.train_nn --model $$m; done
eval:       ; $(PY) -m scripts.run_all_eval
test:       ; $(PY) -m tests.run_tests
```

Makefile recipes need a tab at the start of each recipe line when typed by hand; the one-line `target: ; command` *form above avoids that. On Windows run the* `python -m ...` *commands directly.*

5. Repository structure

Every directory under `src/` and `scripts/` and `tests/` also contains an empty `__init__.py` so that `python -m scripts.x` works from the repository root with imports such as `from src.nn.layers import Dense`.

```
nifty-vol/
|-- README.md                     results summary + how to reproduce (section 10.3)
|-- requirements.txt / requirements-dev.txt / Makefile / .gitignore
|-- .github/
|   |-- copilot-instructions.md   project rules read automatically by Copilot (section 8.1)
|   `-- workflows/tests.yml       CI: tests + gradient checks + banned-import scan
|-- docs/                         this blueprint (PDF + MD), source concept/roadmap docs
|-- data/
|   |-- raw/                      downloaded files, never edited (SOURCES.md tracked)
|   `-- processed/                daily_features.csv, sentiment_daily.csv
|-- src/
|   |-- config.py                 paths, dates, feature lists, hyperparameters
|   |-- io.py                     read features, read/write forecast CSVs, unit helper
|   |-- data/    load_minute.py  load_aux.py  realized_vol.py  features.py  split.py
|   |-- nn/      functional.py  layers.py  losses.py  optim.py  gradcheck.py  trainer.py  utils.py
|   |-- models/  naive.py  har.py  garch.py  common.py  mlp.py  lstm.py  transformer.py
|   |-- sentiment/ tokenize.py  tfidf.py  logreg.py  naive_bayes.py  daily_signal.py
|   `-- eval/    metrics.py  dm_test.py  var_backtest.py  walk_forward.py
|-- scripts/     build_features.py  build_sentiment.py  run_baselines.py  train_nn.py  run_all_eval.py
|-- forecasts/                    one CSV per model: date, target_date, forecast_log_rv
|-- notebooks/                    exploration and plots only (no logic that results depend on)
|-- tests/       run_tests.py  test_gradcheck.py  test_metrics.py  test_no_leakage.py  test_no_banned_imports.py
`-- report/                       figures, tables (CSV) and the write-up
```

Dependency direction (no cycles): `config` <- `io` <- `data` <- `models`/`sentiment` <- `eval` <- `scripts`. `src/eval` never imports `src/models`; it reads forecast CSVs only. This is the roadmap rule that evaluation code never touches model code.

6. Data contracts between modules

6.1 daily_features.csv (one row per trading day)

Column	Definition	Group
date	trading day t	key
target_date	next trading day in the file (t+1)	key
log_rv	log of sum of squared 5-min returns on day t	feature
log_rv_w	log of mean RV over days t-4..t	feature
log_rv_m	log of mean RV over days t-21..t	feature
log_parkinson	log((ln H - ln L)^2 / (4 ln 2) + 1e-12)	feature
ret	ln(close_t / close_t-1)	feature
neg_ret	min(ret, 0)	feature
gap	ln(open_t / close_t-1)	feature
log_vix	log of India VIX close on day t	feature
turnover_chg	log(turnover_t / mean of last 20 turnovers incl. t)	feature
expiry_next	1 if day t+1 is an options expiry day (from calendar file)	feature
dow_0..dow_3	Mon..Thu dummies (Friday = reference)	feature (4)
sent_mean, sent_neg_share, sent_log_count	daily news signal (Phase 5; NaN before)	feature (3)
oc_ret	open-to-close log return of day t (GARCH input)	aux, not a feature
target_log_rv	log RV of day t+1	TARGET
target_oc_ret	open-to-close log return of day t+1 (VaR backtest)	aux, not a feature

6.2 Forecast CSV (forecasts/<model>.csv)

Columns: `date, target_date, forecast_log_rv`. One row per validation and test row. Neural models save the mean of the 5 seeds in `<model>.csv` and each seed in `forecasts/seeds/<model>_seed<k>.csv` so that mean and spread can be reported. Evaluation code joins a forecast CSV to `daily_features.csv` on `date`/`target_date`.

6.3 Alignment rules (the leakage checklist)

- Row t holds features known at the close of day t and the target of day t+1. Windows ending at row t use rows t-T+1..t only; the window's target is row t's target.
- Splits use `target_date`. Scaler mean/std and target mean/std come from **training rows only**.
- Rolling features (weekly, monthly, turnover mean) look backward only; no backward fill; at most 1 day of forward fill when merging VIX and turnover.
- News: a headline is assigned to the first trading day on or after its date. Day t uses headlines dated t or earlier (the archive has no times of day).
- The expiry flag may look at t+1 because the exchange calendar is published in advance.
- `tests/test_no_leakage.py` perturbs all data after a cutoff day and asserts that features on or before the cutoff do not change.

7. Code skeletons and explanations

Conventions used in all code: float64 arrays, shapes written in comments as `(batch, time, features)`, seeded generators passed in explicitly (`rng`), no global random state, type hints on public functions. Each file is shown with an explanation of what it does and why it is written that way.

7.1 Configuration and I/O

`config.py` is the single source of truth for paths, date boundaries, feature lists and hyperparameters. Nothing else hard-codes a date or a feature name, so changing the split or adding a feature is a one-line edit. Hyperparameters follow the roadmap exactly (MLP 70-64-32-1, LSTM 32 hidden, Transformer d=32, 2 heads, 1 block, FFN 64, dropout 0.1).

src/config.py

```
"""Central configuration: paths, dates, feature lists, hyperparameters."""
from dataclasses import dataclass
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
DATA_RAW, DATA_PROC = ROOT / "data" / "raw", ROOT / "data" / "processed"
FORECASTS, REPORT = ROOT / "forecasts", ROOT / "report"

# ---- market constants ------------------------------------------------------
BARS_PER_DAY = 75          # 09:15-15:30 IST in 5-minute bars
MIN_BARS = 60              # drop days with fewer bars (half days, Muhurat session, gaps)
VAR_Z = 2.33               # 99% one-sided normal quantile used for VaR
VAR_LEVEL = 0.01           # expected breach rate

# ---- chronological split (assigned on target_date) -------------------------------------
DATA_START, DATA_END = "2015-01-01", "2023-06-30"
TRAIN_END = "2019-12-31"
VAL_START, VAL_END = "2020-01-01", "2020-12-31"
TEST_START, TEST_END = "2021-01-01", "2023-06-30"

# ---- features --------------------------------------------------------------------------
BASE_FEATURES = [
    "log_rv", "log_rv_w", "log_rv_m", "log_parkinson",        # volatility history / range
    "ret", "neg_ret", "gap",                                  # returns, leverage effect, overnight gap
    "log_vix", "turnover_chg", "expiry_next",                 # expectations, activity, calendar
    "dow_0", "dow_1", "dow_2", "dow_3",                       # Mon..Thu dummies (Fri = reference)
]                                                             # -> 14 inputs
SENT_FEATURES = ["sent_mean", "sent_neg_share", "sent_log_count"]   # -> 17 with sentiment
TARGET, TARGET_OC = "target_log_rv", "target_oc_ret"
HAR_X_EXTRA = ["log_vix", "neg_ret", "gap"]

WINDOW_SEQ = 22            # LSTM / Transformer look-back (one trading month)
WINDOW_MLP = 5             # MLP look-back (one trading week)
SEEDS = (0, 1, 2, 3, 4)

MODEL_NAMES = ["naive", "har", "har_x", "har_x_sent", "garch",
               "mlp", "lstm", "transformer_nosent", "transformer_sent"]


@dataclass(frozen=True)
class TrainCfg:
    lr: float = 1e-3
    weight_decay: float = 1e-4
    batch_size: int = 32
    max_epochs: int = 300
    patience: int = 15
    clip: float = 1.0
    dropout: float = 0.1
    warmup_epochs: int = 0


MLP_CFG = TrainCfg(lr=1e-3, weight_decay=1e-4)
LSTM_CFG = TrainCfg(lr=1e-3, weight_decay=1e-4)
TRF_CFG = TrainCfg(lr=5e-4, weight_decay=1e-3, warmup_epochs=5)
```

`io.py` fixes the file formats so that model code and evaluation code only meet through CSV files. `log_rv_to_vol` is the one place where variance is converted to volatility.

src/io.py

```
"""Reading features and reading/writing forecast files (the shared format between models and eval)."""
import numpy as np
import pandas as pd
from src.config import DATA_PROC, FORECASTS


def log_rv_to_vol(log_rv):
    """log realized VARIANCE -> daily volatility (standard deviation). Use this everywhere."""
    return np.exp(0.5 * np.asarray(log_rv, dtype=float))


def read_features() -> pd.DataFrame:
    df = pd.read_csv(DATA_PROC / "daily_features.csv", parse_dates=["date", "target_date"])
    return df.sort_values("date").reset_index(drop=True)


def write_forecast(name: str, rows: pd.DataFrame, forecast_log_rv, subdir: str = "") -> None:
    """rows must contain the 'date' and 'target_date' of every forecast, in the same order."""
    out = pd.DataFrame({
        "date": rows["date"].to_numpy(),
        "target_date": rows["target_date"].to_numpy(),
        "forecast_log_rv": np.asarray(forecast_log_rv, dtype=float),
    })
    folder = FORECASTS / subdir
    folder.mkdir(parents=True, exist_ok=True)
    out.to_csv(folder / f"{name}.csv", index=False)


def read_forecast(name: str) -> pd.DataFrame:
    return pd.read_csv(FORECASTS / f"{name}.csv", parse_dates=["date", "target_date"])
```

7.2 Data pipeline (Phase 0-1)

These modules turn the raw files into `daily_features.csv`. The order is: load minute data in IST, resample to 5-minute bars, compute RV per day (dropping thin days), merge VIX and turnover, build features, shift the target back one day. Each step is a small pure function so it can be unit tested.

src/data/load_minute.py

```
"""Load 1-minute Nifty OHLC and resample to 5-minute bars within trading hours only."""
import pandas as pd


def load_minute(path, ts_col="date", utc_offset_hours=0.0) -> pd.DataFrame:
    """TODO: set ts_col / rename columns to the real headers of the Kaggle file.
    If timestamps are UTC, pass utc_offset_hours=5.5 so that the first bar is 09:15 IST."""
    df = pd.read_csv(path)
    df.columns = [c.strip().lower() for c in df.columns]
    ts = pd.to_datetime(df[ts_col.lower()]) + pd.to_timedelta(utc_offset_hours, unit="h")
    df = df.set_index(ts).sort_index()[["open", "high", "low", "close"]]
    df = df[~df.index.duplicated(keep="first")]
    return df.between_time("09:15", "15:29")      # drops Muhurat evening session and pre-open noise


def to_5min(df: pd.DataFrame) -> pd.DataFrame:
    """Slots 09:15, 09:20, ..., 15:25 (75 per full day). open=first, close=last of each slot."""
    bars = df.resample("5min").agg({"open": "first", "high": "max", "low": "min", "close": "last"})
    return bars.dropna()
```

src/data/load_aux.py

```
"""Loaders for India VIX, daily turnover and the options-expiry calendar (all indexed by date)."""
import pandas as pd


def _by_date(df: pd.DataFrame, date_col: str) -> pd.DataFrame:
    df = df.copy()
    df.index = pd.to_datetime(df[date_col]).dt.normalize()
    return df[~df.index.duplicated(keep="first")].sort_index()


def load_vix(path, date_col="Date", close_col="Close") -> pd.DataFrame:
    """TODO: confirm column names of the NSE VIX download."""
    df = _by_date(pd.read_csv(path), date_col)
    return df[[close_col]].rename(columns={close_col: "vix_close"})


def load_turnover(path, date_col="Date", turnover_col="Turnover") -> pd.DataFrame:
    """TODO: confirm column names; turnover is in rupee crore."""
    df = _by_date(pd.read_csv(path), date_col)
    return df[[turnover_col]].rename(columns={turnover_col: "turnover"})


def load_expiry_dates(path, date_col="date") -> set:
    """data/raw/expiry_dates.csv: one actual expiry date per row, taken from NSE circulars.
    Never hard-code a weekday: the Nifty expiry weekday changed over the years."""
    return set(pd.to_datetime(pd.read_csv(path)[date_col]).dt.normalize())
```

src/data/realized_vol.py

```
"""Daily realized variance and OHLC from 5-minute bars."""
import numpy as np
import pandas as pd
from src.config import MIN_BARS


def daily_from_bars(bars: pd.DataFrame):
    """Returns (daily frame indexed by date, list of (date, n_bars) dropped days).

    RV_t = sum_i r_{t,i}^2 with r = ln(P_i / P_{i-1}) INSIDE the day. The first return of the day is
    measured from the day's open, so sum(r) = ln(close/open) = the open-to-close return and RV covers the
    same window (09:15-15:30). No return crosses the overnight gap.
    """
    rows, dropped = [], []
    for day, g in bars.groupby(bars.index.date):
        if len(g) < MIN_BARS:
            dropped.append((day, len(g)))
            continue
        closes = g["close"].to_numpy(dtype=float)
        day_open = float(g["open"].iloc[0])
        r = np.diff(np.log(np.r_[day_open, closes]))          # (n_bars,) five-minute log returns
        rows.append({
            "date": pd.Timestamp(day), "n_bars": len(g), "open": day_open,
            "high": float(g["high"].max()), "low": float(g["low"].min()), "close": float(closes[-1]),
            "rv": float(np.sum(r ** 2)),                       # realized VARIANCE
            "oc_ret": float(np.sum(r)),                        # open-to-close log return
        })
    return pd.DataFrame(rows).set_index("date"), dropped
```

src/data/features.py

```
"""Build the one-row-per-day feature table and the next-day target. Only past data feeds a row."""
import numpy as np
import pandas as pd
from src.config import BASE_FEATURES, SENT_FEATURES, TARGET, TARGET_OC


def build_features(daily: pd.DataFrame, vix: pd.DataFrame, turnover: pd.DataFrame, expiry_dates: set):
    df = daily.join(vix[["vix_close"]], how="left").join(turnover[["turnover"]], how="left")
    df[["vix_close", "turnover"]] = df[["vix_close", "turnover"]].ffill(limit=1)   # at most 1 day
    df = df.dropna(subset=["vix_close", "turnover"])                                # drop the rest

    f = pd.DataFrame(index=df.index)
    prev_close = df["close"].shift(1)
    f["log_rv"] = np.log(df["rv"])
    f["log_rv_w"] = np.log(df["rv"].rolling(5).mean())            # days t-4..t
    f["log_rv_m"] = np.log(df["rv"].rolling(22).mean())           # days t-21..t
    f["log_parkinson"] = np.log(np.log(df["high"] / df["low"]) ** 2 / (4 * np.log(2)) + 1e-12)
    f["ret"] = np.log(df["close"] / prev_close)
    f["neg_ret"] = f["ret"].clip(upper=0.0)                       # leverage-effect input
    f["gap"] = np.log(df["open"] / prev_close)                    # overnight gap
    f["log_vix"] = np.log(df["vix_close"])
    f["turnover_chg"] = np.log(df["turnover"] / df["turnover"].rolling(20).mean())

    next_day = pd.Series(df.index, index=df.index).shift(-1)      # t+1 (NaT on the last row)
    f["expiry_next"] = next_day.isin(expiry_dates).astype(float)
    for k in range(4):                                            # Mon..Thu dummies
        f[f"dow_{k}"] = (df.index.dayofweek == k).astype(float)
    for c in SENT_FEATURES:                                       # filled in Phase 5
        f[c] = np.nan

    f["oc_ret"] = df["oc_ret"]
    f[TARGET] = f["log_rv"].shift(-1)                             # row t holds day t+1's value
    f[TARGET_OC] = df["oc_ret"].shift(-1)
    f["target_date"] = next_day
    f = f.dropna(subset=BASE_FEATURES + [TARGET, TARGET_OC, "target_date"])   # warm-up rows + last row
    f.index.name = "date"
    return f.reset_index()
```

Why `rolling(5)` **is leak-free:** pandas rolling windows end at the current row, so day t's weekly average uses days t-4..t only. The target is the only quantity shifted backwards (`shift(-1)`), and it is stored in its own column that is never in BASE_FEATURES.

src/data/split.py

```
"""Chronological split, train-only scaling, and window building."""
import numpy as np
import pandas as pd
from src.config import (TRAIN_END, VAL_START, VAL_END, TEST_START, TEST_END)


def split_masks(df: pd.DataFrame) -> dict:
    """Boolean masks over df rows, assigned by target_date (so no train target lies in validation)."""
    td = df["target_date"]
    return {
        "train": (td <= TRAIN_END).to_numpy(),
        "val": ((td >= VAL_START) & (td <= VAL_END)).to_numpy(),
        "test": ((td >= TEST_START) & (td <= TEST_END)).to_numpy(),
    }


class Scaler:
    """Standardise columns with statistics from the TRAIN rows only, then apply unchanged elsewhere."""
    def fit(self, X: np.ndarray) -> "Scaler":
        self.mean = X.mean(axis=0)
        self.std = X.std(axis=0) + 1e-8
        return self

    def transform(self, X: np.ndarray) -> np.ndarray:
        return (X - self.mean) / self.std


def make_windows(X: np.ndarray, y: np.ndarray, T: int):
    """X: (n, F) scaled features in date order; y: (n,). Window i covers rows i-T+1..i (past only) and
    its target is y[i] (day i+1). Returns Xw (m, T, F), yw (m,), end_idx (m,) = row index of each window end.
    The first T-1 rows cannot form a window. Windows may reach back into earlier splits (past history is fine)."""
    end_idx = np.arange(T - 1, len(X))
    Xw = np.stack([X[i - T + 1: i + 1] for i in end_idx])          # (m, T, F)
    return Xw, y[end_idx], end_idx
```

7.3 NumPy neural-network core (Phase 3)

This is the library every neural model sits on. Design: every building block is a `Layer` with `forward(x, training)`, `backward(dout)` and `own_params()` returning `(value, grad)` array pairs. `parameters()` collects them recursively through `children()`, so a Transformer is just a tree of layers and **one training loop serves all models**. Gradients accumulate into the grad arrays; call `zero_grad()` before each backward pass.

src/nn/functional.py

```
"""Stateless math helpers (forward and the matching backward)."""
import numpy as np


def sigmoid(x):
    return 0.5 * (1.0 + np.tanh(0.5 * x))           # numerically stable, no overflow warnings


def softmax(x, axis=-1):
    z = x - x.max(axis=axis, keepdims=True)         # subtract max: same result, no exp overflow
    e = np.exp(z)
    return e / e.sum(axis=axis, keepdims=True)


def softmax_backward(p, dp, axis=-1):
    """p = softmax(s). dL/ds = p * (dL/dp - sum(dL/dp * p))."""
    return p * (dp - (dp * p).sum(axis=axis, keepdims=True))
```

src/nn/layers.py

```
"""Layers with hand-written forward/backward. All arrays float64. Shapes are noted in comments."""
import numpy as np
from src.nn.functional import softmax, softmax_backward

_C = np.sqrt(2.0 / np.pi)


class Layer:
    def forward(self, x, training=True): raise NotImplementedError
    def backward(self, dout): raise NotImplementedError
    def own_params(self): return []                # list of (value, grad) arrays owned by THIS layer
    def children(self): return []                  # sub-layers

    def parameters(self):
        ps = list(self.own_params())
        for c in self.children():
            ps += c.parameters()
        return ps

    def zero_grad(self):
        for _, g in self.parameters():
            g[...] = 0.0


class Dense(Layer):
    """y = x W + b, applied on the last axis, so x may be (B, in) or (B, T, in)."""
    def __init__(self, n_in, n_out, rng, init="he"):
        std = np.sqrt((2.0 if init == "he" else 1.0) / n_in)        # He for ReLU/GELU, Xavier otherwise
        self.n_in = n_in
        self.W, self.b = rng.normal(0.0, std, (n_in, n_out)), np.zeros(n_out)
        self.dW, self.db = np.zeros_like(self.W), np.zeros_like(self.b)

    def own_params(self): return [(self.W, self.dW), (self.b, self.db)]

    def forward(self, x, training=True):
        self.x_shape, self.x2 = x.shape, x.reshape(-1, self.n_in)   # cache input for backward
        return (self.x2 @ self.W + self.b).reshape(*x.shape[:-1], -1)

    def backward(self, dout):
        d2 = dout.reshape(-1, self.W.shape[1])
        self.dW += self.x2.T @ d2                                   # dL/dW = X^T dY
        self.db += d2.sum(axis=0)                                   # dL/db = sum over batch
        return (d2 @ self.W.T).reshape(self.x_shape)                # dL/dX = dY W^T


class ReLU(Layer):
    def forward(self, x, training=True):
        self.mask = x > 0
        return x * self.mask
    def backward(self, dout): return dout * self.mask


class GELU(Layer):
    """tanh approximation: 0.5 x (1 + tanh(c (x + 0.044715 x^3)))."""
    def forward(self, x, training=True):
        self.x = x
        self.t = np.tanh(_C * (x + 0.044715 * x ** 3))
        return 0.5 * x * (1.0 + self.t)

    def backward(self, dout):
        x, t = self.x, self.t
        dt = (1.0 - t ** 2) * _C * (1.0 + 3 * 0.044715 * x ** 2)
        return dout * (0.5 * (1.0 + t) + 0.5 * x * dt)


class Dropout(Layer):
    """Inverted dropout: scale kept units by 1/(1-rate) in training; identity at test time."""
    def __init__(self, rate, rng):
        self.rate, self.rng, self.mask = rate, rng, None

    def forward(self, x, training=True):
        if not training or self.rate == 0.0:
            self.mask = None
            return x
        self.mask = (self.rng.random(x.shape) >= self.rate) / (1.0 - self.rate)
        return x * self.mask

    def backward(self, dout): return dout if self.mask is None else dout * self.mask


class Flatten(Layer):
    def forward(self, x, training=True):
        self.shape = x.shape
        return x.reshape(x.shape[0], -1)
    def backward(self, dout): return dout.reshape(self.shape)


class LayerNorm(Layer):
```

```
    """Normalise each vector over the last axis to mean 0 / var 1, then learned scale and shift."""
    def __init__(self, d, eps=1e-5):
        self.eps = eps
        self.gamma, self.beta = np.ones(d), np.zeros(d)
        self.dgamma, self.dbeta = np.zeros(d), np.zeros(d)

    def own_params(self): return [(self.gamma, self.dgamma), (self.beta, self.dbeta)]

    def forward(self, x, training=True):
        mu, var = x.mean(axis=-1, keepdims=True), x.var(axis=-1, keepdims=True)
        self.inv = 1.0 / np.sqrt(var + self.eps)                    # 1 / sigma
        self.xhat = (x - mu) * self.inv
        return self.gamma * self.xhat + self.beta

    def backward(self, dout):
        axes = tuple(range(dout.ndim - 1))
        self.dgamma += (dout * self.xhat).sum(axis=axes)
        self.dbeta += dout.sum(axis=axes)
        dxh, N = dout * self.gamma, dout.shape[-1]
        return self.inv / N * (N * dxh - dxh.sum(axis=-1, keepdims=True)
                               - self.xhat * (dxh * self.xhat).sum(axis=-1, keepdims=True))


class Softmax(Layer):                                               # mainly used for gradient checks
    def forward(self, x, training=True):
        self.p = softmax(x)
        return self.p
    def backward(self, dout): return softmax_backward(self.p, dout)


class Sequential(Layer):
    def __init__(self, layers): self.layers = list(layers)
    def children(self): return self.layers

    def forward(self, x, training=True):
        for l in self.layers:
            x = l.forward(x, training)
        return x

    def backward(self, dout):
        for l in reversed(self.layers):
            dout = l.backward(dout)
        return dout
```

Backward formulas used (from the concepts document): Dense `dW = X^T dY`, `db = sum dY`, `dX = dY W^T`; softmax `dS = P * (dP - rowsum(dP * P))`; layer norm `dx = (1/(N sigma)) (N dxhat - sum dxhat - xhat sum(dxhat * xhat))` with `dxhat = dy * gamma`. Inverted dropout stores its mask and multiplies the gradient by it.

src/nn/losses.py

```
"""Losses return a scalar from forward() and the gradient w.r.t. predictions from backward()."""
import numpy as np
from src.nn.functional import softmax


class MSELoss:
    def forward(self, pred, target):
        self.diff = pred - target.reshape(pred.shape)
        return float(np.mean(self.diff ** 2))

    def backward(self):
        return 2.0 * self.diff / self.diff.size                     # dL/dpred = 2/N (pred - target)


class SoftmaxCrossEntropy:
    """logits (N, C), integer labels (N,). Gradient w.r.t. logits is (p - onehot) / N."""
    def forward(self, logits, labels):
        self.p, self.labels = softmax(logits), labels
        return float(-np.mean(np.log(self.p[np.arange(len(labels)), labels] + 1e-12)))

    def backward(self):
        g = self.p.copy()
        g[np.arange(len(self.labels)), self.labels] -= 1.0
        return g / len(self.labels)
```

src/nn/optim.py

```
"""Adam (optionally AdamW-style decoupled weight decay) and gradient-norm clipping."""
import numpy as np


class Adam:
    def __init__(self, params, lr=1e-3, betas=(0.9, 0.999), eps=1e-8, weight_decay=0.0):
        self.params, self.lr, self.betas, self.eps, self.wd = params, lr, betas, eps, weight_decay
        self.m = [np.zeros_like(w) for w, _ in params]              # running mean of gradient
        self.v = [np.zeros_like(w) for w, _ in params]              # running mean of squared gradient
        self.t = 0

    def step(self):
        self.t += 1
        b1, b2 = self.betas
        for (w, g), m, v in zip(self.params, self.m, self.v):
            m *= b1; m += (1 - b1) * g
            v *= b2; v += (1 - b2) * g * g
            m_hat, v_hat = m / (1 - b1 ** self.t), v / (1 - b2 ** self.t)   # bias correction
            if self.wd and w.ndim >= 2:                              # decay weight matrices, not biases/LN
                w -= self.lr * self.wd * w
            w -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)


def clip_grad_norm(params, max_norm=1.0) -> float:
    total = float(np.sqrt(sum(float(np.sum(g * g)) for _, g in params)))
    if total > max_norm:
        scale = max_norm / (total + 1e-12)
        for _, g in params:
            g *= scale
    return total
```

src/nn/gradcheck.py

```
"""Finite-difference gradient checking. Run on EVERY layer before it is used in training."""
import numpy as np


def _num_grad(f, arr, eps):
    g = np.zeros_like(arr)
    for idx in np.ndindex(arr.shape):
        old = arr[idx]
        arr[idx] = old + eps; fp = f()
        arr[idx] = old - eps; fm = f()
        arr[idx] = old
        g[idx] = (fp - fm) / (2 * eps)                              # central difference
    return g


def rel_err(a, b):
    """Relative error ||a-b|| / (||a|| + ||b||). If both gradients are ~0 (e.g. the attention KEY bias, whose
    true gradient is exactly zero because softmax ignores a constant shift) compare absolutely instead."""
    na, nb = np.linalg.norm(a), np.linalg.norm(b)
    if na + nb < 1e-8:
        return float(np.linalg.norm(a - b))
    return float(np.linalg.norm(a - b) / (na + nb))


def gradcheck_layer(layer, x, rng, eps=1e-5):
    """Scalar objective L = sum(layer(x) * w) with random w. Returns {name: relative error} for the
    input and every parameter. Pass if all < 1e-6 (float64, training=False so dropout is inactive)."""
    x = np.asarray(x, dtype=np.float64).copy()
    w = rng.standard_normal(layer.forward(x, training=False).shape)
    f = lambda: float(np.sum(layer.forward(x, training=False) * w))
    layer.zero_grad()
    layer.forward(x, training=False)
    dx = layer.backward(w)                                          # analytic gradients
    errs = {"input": rel_err(dx, _num_grad(f, x, eps))}
    for k, (val, grad) in enumerate(layer.parameters()):
        errs[f"param{k}"] = rel_err(grad, _num_grad(f, val, eps))
    return errs


def gradcheck_loss(loss, pred, target, eps=1e-5):
    pred = pred.astype(np.float64).copy()
    f = lambda: loss.forward(pred, target)
    f()
    return rel_err(loss.backward(), _num_grad(f, pred, eps))
```

src/nn/trainer.py

```
"""One training loop for every model: mini-batches, Adam, clipping, warm-up, early stopping."""
import numpy as np
from src.nn.losses import MSELoss
from src.nn.optim import Adam, clip_grad_norm


def fit(model, X_tr, y_tr, X_va, y_va, cfg, rng):
    """Shuffles training WINDOWS each epoch (the date split itself is unchanged). Restores the weights of
    the epoch with the lowest validation loss. Returns the loss history."""
    loss, params = MSELoss(), model.parameters()
    opt = Adam(params, lr=cfg.lr, weight_decay=cfg.weight_decay)
    best, best_w, bad, hist = np.inf, None, 0, []
    for epoch in range(cfg.max_epochs):
        if cfg.warmup_epochs:                                       # linear warm-up of the learning rate
            opt.lr = cfg.lr * min(1.0, (epoch + 1) / cfg.warmup_epochs)
        order, tr_sum = rng.permutation(len(X_tr)), 0.0
        for s in range(0, len(order), cfg.batch_size):
            b = order[s:s + cfg.batch_size]
            model.zero_grad()
            pred = model.forward(X_tr[b], training=True)            # (batch, 1)
            tr_sum += loss.forward(pred, y_tr[b]) * len(b)
            model.backward(loss.backward())
            clip_grad_norm(params, cfg.clip)
            opt.step()
        va = loss.forward(model.forward(X_va, training=False), y_va)
        hist.append((tr_sum / len(X_tr), va))
        if va < best - 1e-9:
            best, bad, best_w = va, 0, [w.copy() for w, _ in params]
        else:
            bad += 1
            if bad >= cfg.patience:
                break
    for (w, _), saved in zip(params, best_w):                       # restore best epoch
        w[...] = saved
    return hist
```

src/nn/utils.py

```
def count_params(model) -> int:
    """Keep LSTM and Transformer within ~2x of each other so the comparison is fair."""
    return int(sum(v.size for v, _ in model.parameters()))
```

7.4 Classical baselines (Phase 2)

Naive, HAR-RV and GARCH are the bar every neural network must clear. HAR is a linear regression solved exactly with `np.linalg.solve` (never an explicit inverse). GARCH(1,1) is fitted by maximum likelihood: the three parameters are unconstrained numbers that are transformed so that omega > 0, 0 < alpha + beta < 1 and alpha, beta >= 0; the gradient is a central finite difference (6 likelihood evaluations per step, acceptable for 3 parameters) fed to our own Adam. Returns are multiplied by 100 because raw daily returns (~0.01) make omega about 1e-6 and cause numerical trouble.

src/models/naive.py

```
import numpy as np


def naive_forecast(df) -> np.ndarray:
    """Rung 0: tomorrow's log RV = today's log RV. Anything that cannot beat this is useless."""
    return df["log_rv"].to_numpy()
```

src/models/har.py

```
"""HAR-RV (Corsi 2009): log RV_{t+1} = b0 + bd*logRV_t + bw*logRV_week + bm*logRV_month (+ extra columns)."""
import numpy as np


def har_design(df, extra=()):
    cols = ["log_rv", "log_rv_w", "log_rv_m", *extra]
    return np.column_stack([np.ones(len(df)), df[list(cols)].to_numpy(dtype=float)])   # (n, k) with intercept


def fit_har(X, y):
    """OLS: beta = (X'X)^-1 X'y, solved without forming the inverse (more stable)."""
    return np.linalg.solve(X.T @ X, X.T @ y)


def predict_har(X, beta):
    return X @ beta
```

src/models/garch.py

```
"""GARCH(1,1) by Gaussian maximum likelihood, in pure NumPy. Work in PERCENT returns."""
import numpy as np
from src.nn.functional import sigmoid
from src.nn.optim import Adam


def unpack(theta):
    omega = np.exp(theta[0])                    # > 0
    p = sigmoid(theta[1])                       # persistence alpha+beta in (0, 1)
    alpha = p * sigmoid(theta[2])               # share of persistence that reacts to shocks
    return omega, alpha, p - alpha              # (omega, alpha, beta)


def garch_filter(theta, r, s2_start):
    """r: (n,) returns. Returns s2 of length n+1: s2[t] = variance of day t, s2[t+1] = forecast for day t+1
    made with information up to day t (this alignment is what the forecast CSV needs)."""
    omega, alpha, beta = unpack(theta)
    s2 = np.empty(len(r) + 1)
    s2[0] = s2_start
    for t in range(len(r)):
        s2[t + 1] = omega + alpha * r[t] ** 2 + beta * s2[t]
    return s2


def garch_nll(theta, r):
    s2 = garch_filter(theta, r, r.var())[:-1]
    return 0.5 * np.sum(np.log(2 * np.pi) + np.log(s2) + r ** 2 / s2)


def _num_grad(f, theta, eps=1e-4):
    g = np.zeros_like(theta)
    for i in range(len(theta)):
        d = np.zeros_like(theta); d[i] = eps
        g[i] = (f(theta + d) - f(theta - d)) / (2 * eps)
    return g


def fit_garch(r_train_pct, steps=400, lr=0.05):
    """Minimise the mean negative log-likelihood over (a, b, c) with Adam + numerical gradient.
    Start: persistence ~0.97, alpha share ~0.1, omega set so the long-run variance equals the sample variance."""
    theta = np.array([np.log(r_train_pct.var() * 0.03), 3.5, -2.2])
    grad = np.zeros(3)
    opt = Adam([(theta, grad)], lr=lr)
    f = lambda th: garch_nll(th, r_train_pct) / len(r_train_pct)
    for _ in range(steps):
        grad[...] = _num_grad(f, theta)
        opt.step()
    return theta


def garch_log_rv_forecast(theta, r_all_pct, s2_start):
    """Log-variance forecast (decimal units, same as log RV) for the day after each r_all value."""
    s2 = garch_filter(theta, r_all_pct, s2_start)
    return np.log(s2[1:] / 1e4)                 # percent^2 -> decimal^2
```

Sanity checks to print in the README: HAR coefficients are positive and sum to slightly below 1; GARCH persistence alpha + beta lies between about 0.90 and 0.995; HAR beats Naive on validation QLIKE.

7.5 Shared model plumbing and the MLP (Phase 3)

`common.py` prepares scaled windows, trains one model for several seeds, converts predictions back to log RV units and writes forecast CSVs. The three neural models only differ in the `build` function they pass in, which is what keeps the comparison fair (same data, same splits, same seeds, same loop).

src/models/common.py

```
"""Prepare windows, train over seeds, write forecasts. Shared by MLP, LSTM and both Transformers."""
import numpy as np
from src.config import TARGET, SEEDS, TRAIN_END, VAL_START, VAL_END, TEST_START, TEST_END
from src.data.split import Scaler, make_windows, split_masks
from src.io import read_features, write_forecast
from src.nn.trainer import fit
from src.nn.utils import count_params


def prepare(features, window):
    df = read_features()
    m = split_masks(df)
    X = Scaler().fit(df.loc[m["train"], features].to_numpy()).transform(df[features].to_numpy())
    mu, sd = df.loc[m["train"], TARGET].mean(), df.loc[m["train"], TARGET].std()   # TRAIN-only target stats
    y = (df[TARGET].to_numpy() - mu) / sd                          # standardised target (see section 2.5)
    Xw, yw, end_idx = make_windows(X, y, window)
    td = df["target_date"].iloc[end_idx].reset_index(drop=True)
    sets = {"train": (td <= TRAIN_END), "val": (td >= VAL_START) & (td <= VAL_END),
            "test": (td >= TEST_START) & (td <= TEST_END)}
    return df, Xw, yw, end_idx, {k: v.to_numpy() for k, v in sets.items()}, (mu, sd)


def run_model(name, features, window, build, cfg, seeds=SEEDS):
    """build(n_features, window, rng) -> model. Trains one model per seed; writes the seed-mean forecast to
    forecasts/<name>.csv and each seed to forecasts/seeds/<name>_seed<k>.csv (val + test rows only)."""
    df, Xw, yw, end_idx, S, (mu, sd) = prepare(features, window)
    keep = S["val"] | S["test"]
    rows = df.iloc[end_idx[keep]]
    all_preds = []
    for seed in seeds:
        rng = np.random.default_rng(seed)
        model = build(len(features), window, rng)
        hist = fit(model, Xw[S["train"]], yw[S["train"]], Xw[S["val"]], yw[S["val"]], cfg, rng)
        pred = model.forward(Xw[keep], training=False).ravel() * sd + mu      # back to log RV units
        write_forecast(f"{name}_seed{seed}", rows, pred, subdir="seeds")
        all_preds.append(pred)
        best_val = min(h[1] for h in hist)
        print(f"{name} seed={seed} params={count_params(model)} epochs={len(hist)} best_val={best_val:.4f}")
    write_forecast(name, rows, np.mean(all_preds, axis=0))
```

src/models/mlp.py

```
"""Rung 3: MLP on the last 5 days flattened (5 x 14 = 70 inputs): 70 -> 64 -> ReLU -> dropout -> 32 -> ReLU -> 1."""
from src.nn.layers import Sequential, Flatten, Dense, ReLU, Dropout


def build_mlp(n_features, window, rng, p=0.1):
    return Sequential([
        Flatten(),                                  # (B, T, F) -> (B, T*F)
        Dense(n_features * window, 64, rng), ReLU(), Dropout(p, rng),
        Dense(64, 32, rng), ReLU(),
        Dense(32, 1, rng, init="xavier"),           # linear output: tomorrow's standardised log RV
    ])
```

7.6 LSTM (Phase 4)

The LSTM reads 22 days one by one and predicts from its final hidden state. One stacked weight matrix of shape (n_in + hidden, 4 x hidden) computes all four gate pre-activations in one matrix multiply, then it is sliced into forget, input, output and candidate. Backpropagation through time walks from step T back to 1: at each step it receives `dh` and `dc`, accumulates weight gradients (shared across steps, so +=), and passes `dh` and `dc = dc_total * f` to the previous step. The forget-gate bias starts at 1 so memory is kept by default. With 14 inputs and 32 hidden units there are 6,016 LSTM parameters, 6,049 with the head.

src/models/lstm.py

```
"""Rung 4: LSTM(14 -> 32) -> last hidden state -> dropout -> Dense(32 -> 1). Gate order: f, i, o, g."""
import numpy as np
from src.nn.functional import sigmoid
from src.nn.layers import Layer, Sequential, Dense, Dropout


class LSTM(Layer):
    def __init__(self, n_in, n_hid, rng):
        self.n_in, self.H = n_in, n_hid
        self.W = rng.normal(0.0, np.sqrt(1.0 / (n_in + n_hid)), (n_in + n_hid, 4 * n_hid))  # stacked
        self.b = np.zeros(4 * n_hid)
        self.b[:n_hid] = 1.0                                        # forget-gate bias = 1
        self.dW, self.db = np.zeros_like(self.W), np.zeros_like(self.b)

    def own_params(self): return [(self.W, self.dW), (self.b, self.db)]

    def forward(self, x, training=True):                            # x: (B, T, n_in) -> (B, H)
        B, T, _ = x.shape
        H = self.H
        h, c = np.zeros((B, H)), np.zeros((B, H))
        self.cache, self.B, self.T = [], B, T
        for t in range(T):
            z_in = np.concatenate([x[:, t, :], h], axis=1)          # (B, n_in + H)
            a = z_in @ self.W + self.b                              # (B, 4H) all gates at once
            f, i = sigmoid(a[:, :H]), sigmoid(a[:, H:2 * H])
            o, g = sigmoid(a[:, 2 * H:3 * H]), np.tanh(a[:, 3 * H:])
            c_new = f * c + i * g                                   # cell state: updated by ADDITION
            tc = np.tanh(c_new)
            self.cache.append((z_in, f, i, o, g, c, tc))            # everything backward needs
            h, c = o * tc, c_new
        return h

    def backward(self, dh_last):                                    # dh_last: (B, H) -> dx: (B, T, n_in)
        H, dh = self.H, dh_last
        dc = np.zeros_like(dh)
        dx = np.zeros((self.B, self.T, self.n_in))
        for t in reversed(range(self.T)):
            z_in, f, i, o, g, c_prev, tc = self.cache[t]
            do = dh * tc
            dc = dc + dh * o * (1.0 - tc ** 2)                      # total gradient reaching c_t
            df, di, dg = dc * c_prev, dc * g, dc * i
            da = np.concatenate([df * f * (1 - f), di * i * (1 - i),
                                 do * o * (1 - o), dg * (1 - g ** 2)], axis=1)   # through the gate non-linearities
            self.dW += z_in.T @ da                                  # SUM over time steps (shared weights)
            self.db += da.sum(axis=0)
            dz = da @ self.W.T
            dx[:, t, :], dh = dz[:, :self.n_in], dz[:, self.n_in:]
            dc = dc * f                                             # dc_{t-1} = dc_t * f_t
        return dx


def build_lstm(n_features, window, rng, hidden=32, p=0.1):
    return Sequential([LSTM(n_features, hidden, rng), Dropout(p, rng), Dense(hidden, 1, rng, init="xavier")])
```

Gradient-check the full unrolled layer on the tiny case from the roadmap: 2 inputs, 3 hidden units, 4 steps, batch 2 (section 9.1). Clipping at norm 1.0 is applied by the trainer.

7.7 Time-series Transformer (Phase 6)

A language Transformer embeds word ids; here each position is a **trading day**, so the embedding lookup is replaced by a linear projection of that day's 14 (or 17) features to d = 32, plus a learned positional embedding (22 x 32) because attention alone is order-blind. No decoder, no causal mask: all 22 days are in the past relative to the target, so letting any day attend to any other leaks nothing (the leakage risk is in window construction, not in attention). The encoder block is **pre-norm**: `Z = H + MHA(LN(H))`, `H' = Z + FFN(LN(Z))` with FFN d -> 2d -> d using GELU. The output is the mean over the 22 days, then Dense to one number. Roughly 9,800 parameters, comparable to the LSTM.

Attention, step by step: Q = XWq, K = XWk, V = XWv are split into heads, scores = QK^T / sqrt(d_k), softmax along each row gives weights that sum to 1, output = weights x V, heads are concatenated and mixed by Wo. The averaged 22 x 22 weight grid is the interpretability plot (`last_attn` is stored for that).

src/models/transformer.py

```
"""Rung 5: encoder-only time-series Transformer, from scratch (all weights random-init, trained here)."""
import numpy as np
from src.nn.functional import softmax, softmax_backward
from src.nn.layers import Layer, Sequential, Dense, GELU, Dropout, LayerNorm


class MultiHeadAttention(Layer):
    def __init__(self, d, heads, rng):
        assert d % heads == 0
        self.d, self.h, self.dk = d, heads, d // heads
        self.Wq, self.Wk, self.Wv, self.Wo = (Dense(d, d, rng, init="xavier") for _ in range(4))
        self.last_attn = None                                   # (B, heads, T, T) for the heatmap

    def children(self): return [self.Wq, self.Wk, self.Wv, self.Wo]

    def _split(self, x):                                        # (B, T, d) -> (B, h, T, dk)
        B, T, _ = x.shape
        return x.reshape(B, T, self.h, self.dk).transpose(0, 2, 1, 3)

    def _merge(self, x):                                        # (B, h, T, dk) -> (B, T, d)
        B, h, T, dk = x.shape
        return x.transpose(0, 2, 1, 3).reshape(B, T, h * dk)

    def forward(self, x, training=True):
        q, k, v = (self._split(W.forward(x)) for W in (self.Wq, self.Wk, self.Wv))
        s = (q @ k.transpose(0, 1, 3, 2)) / np.sqrt(self.dk)        # (B, h, T, T) relevance of day j to day i
        p = softmax(s, axis=-1)                                     # rows sum to 1
        self.cache, self.last_attn = (q, k, v, p), p
        return self.Wo.forward(self._merge(p @ v))                  # blend values, concat heads, mix with Wo

    def backward(self, dout):
        q, k, v, p = self.cache
        dctx = self._split(self.Wo.backward(dout))                  # grad of (p @ v)
        dp = dctx @ v.transpose(0, 1, 3, 2)                         # dP = dA V^T
        dv = p.transpose(0, 1, 3, 2) @ dctx                         # dV = P^T dA
        ds = softmax_backward(p, dp) / np.sqrt(self.dk)             # through softmax and the 1/sqrt(dk) scale
        dq, dk_ = ds @ k, ds.transpose(0, 1, 3, 2) @ q              # dQ = dS K ; dK = dS^T Q
        return (self.Wq.backward(self._merge(dq)) + self.Wk.backward(self._merge(dk_))
                + self.Wv.backward(self._merge(dv)))                # three paths into x: add gradients


class EncoderBlock(Layer):
    def __init__(self, d, heads, rng, ff_mult=2, p=0.1):
        self.ln1, self.mha = LayerNorm(d), MultiHeadAttention(d, heads, rng)
        self.ln2 = LayerNorm(d)
        self.ffn = Sequential([Dense(d, ff_mult * d, rng), GELU(), Dense(ff_mult * d, d, rng, init="xavier")])
        self.drop1, self.drop2 = Dropout(p, rng), Dropout(p, rng)

    def children(self): return [self.ln1, self.mha, self.ln2, self.ffn, self.drop1, self.drop2]

    def forward(self, x, training=True):
        z = x + self.drop1.forward(self.mha.forward(self.ln1.forward(x), training), training)   # residual 1
        return z + self.drop2.forward(self.ffn.forward(self.ln2.forward(z), training), training)  # residual 2

    def backward(self, dout):                                   # residual branches ADD their gradients
        dz = dout + self.ln2.backward(self.ffn.backward(self.drop2.backward(dout)))
        return dz + self.ln1.backward(self.mha.backward(self.drop1.backward(dz)))


class TimeSeriesTransformer(Layer):
    def __init__(self, n_features, window, rng, d=32, heads=2, blocks=1, ff_mult=2, p=0.1):
        self.in_proj = Dense(n_features, d, rng, init="xavier")     # replaces the word-embedding lookup
        self.pos = rng.normal(0.0, 0.02, (window, d))               # learned positional embedding
        self.dpos = np.zeros_like(self.pos)
        self.blocks = [EncoderBlock(d, heads, rng, ff_mult, p) for _ in range(blocks)]
        self.drop, self.head = Dropout(p, rng), Dense(d, 1, rng, init="xavier")

    def own_params(self): return [(self.pos, self.dpos)]
    def children(self): return [self.in_proj, *self.blocks, self.drop, self.head]

    def forward(self, x, training=True):                           # x: (B, T, F) -> (B, 1)
        h = self.in_proj.forward(x, training) + self.pos[None, :, :]
        for blk in self.blocks:
            h = blk.forward(h, training)
        self.T = h.shape[1]
        return self.head.forward(self.drop.forward(h.mean(axis=1), training), training)   # mean-pool days

    def backward(self, dout):
        dpool = self.drop.backward(self.head.backward(dout))       # (B, d)
        dh = np.repeat(dpool[:, None, :] / self.T, self.T, axis=1)   # mean-pool backward: spread equally
        for blk in reversed(self.blocks):
            dh = blk.backward(dh)
        self.dpos += dh.sum(axis=0)                                 # positional embedding is shared over batch
        return self.in_proj.backward(dh)

    def mean_attention(self):
        """(T, T) attention weights of the first block averaged over batch and heads (for the plot)."""
        return self.blocks[0].mha.last_attn.mean(axis=(0, 1))
```

```
def build_transformer(n_features, window, rng, d=32, heads=2, blocks=1, p=0.1):
    return TimeSeriesTransformer(n_features, window, rng, d=d, heads=heads, blocks=blocks, p=p)
```

Headline experiment: train `transformer_nosent` (14 inputs) and `transformer_sent` (17 inputs) with the identical architecture, hyperparameters and seeds, then compare QLIKE with the Diebold-Mariano test. A clear 'no improvement' is a valid result and should be reported as such. The small hyperparameter search is d in {16, 32}, blocks in {1, 2}, lr in {3e-4, 1e-3} judged on validation QLIKE over 3 seeds; freeze it before touching the test set.

7.8 News sentiment from scratch (Phase 5)

No pretrained language model is used. The classifier is a bag-of-words multinomial logistic regression (softmax regression) trained on Financial PhraseBank, reusing our own `Dense` layer, `SoftmaxCrossEntropy` and `Adam`. Pipeline: tokenise (lowercase, digits -> `num`, strip punctuation, add bigrams) -> vocabulary (min count 3, **train split only**) -> TF-IDF with `idf = log((1+N)/(1+df)) + 1`, L2-normalised -> classifier. A Naive Bayes baseline with add-one smoothing is the sanity check: logistic regression should not lose to it. Then evaluate on the 400 labelled Economic Times headlines and report both accuracies.

src/sentiment/tokenize.py

```
import re

_NUM = re.compile(r"\d+(?:[.,]\d+)*")
_PUNCT = re.compile(r"[^a-z0-9\s]")


def tokenize(text: str, bigrams: bool = True):
    t = _NUM.sub(" num ", text.lower())             # numbers -> 'num' token
    t = _PUNCT.sub(" ", t)
    words = t.split()
    if not bigrams:
        return words
    return words + [f"{a}_{b}" for a, b in zip(words, words[1:])]   # bigrams keep 'not good' != 'good'
```

src/sentiment/tfidf.py

```
import numpy as np
from collections import Counter


class Tfidf:
    def fit(self, docs, min_count=3):
        """docs: list of token lists from the TRAIN split only."""
        total = Counter(tok for d in docs for tok in d)
        self.vocab = {w: i for i, w in enumerate(sorted(w for w, c in total.items() if c >= min_count))}
        df = np.zeros(len(self.vocab))
        for d in docs:
            for tok in set(d):
                if tok in self.vocab:
                    df[self.vocab[tok]] += 1
        self.idf = np.log((1 + len(docs)) / (1 + df)) + 1.0
        return self

    def counts(self, docs):
        X = np.zeros((len(docs), len(self.vocab)))
        for r, d in enumerate(docs):
            for tok in d:
                j = self.vocab.get(tok)
                if j is not None:
                    X[r, j] += 1.0
        return X

    def transform(self, docs):
        X = self.counts(docs) * self.idf                                # tf x idf
        return X / np.maximum(np.linalg.norm(X, axis=1, keepdims=True), 1e-12)   # L2 normalise
```

src/sentiment/logreg.py

```
"""Softmax regression: TF-IDF vector -> 3 scores (0=negative, 1=neutral, 2=positive)."""
import numpy as np
from src.nn.layers import Dense
from src.nn.losses import SoftmaxCrossEntropy
from src.nn.optim import Adam
from src.nn.functional import softmax


class SoftmaxRegression:
    def __init__(self, n_features, rng):
        self.dense, self.loss, self.rng = Dense(n_features, 3, rng, init="xavier"), SoftmaxCrossEntropy(), rng

    def fit(self, X, y, epochs=60, lr=5e-3, weight_decay=1e-3, batch=64):
        opt = Adam(self.dense.parameters(), lr=lr, weight_decay=weight_decay)
        for _ in range(epochs):
            order = self.rng.permutation(len(X))
            for s in range(0, len(X), batch):
                b = order[s:s + batch]
                self.dense.zero_grad()
                self.loss.forward(self.dense.forward(X[b]), y[b])
                self.dense.backward(self.loss.backward())
                opt.step()
        return self

    def predict_proba(self, X):
        return softmax(self.dense.forward(X, training=False))

    def top_words(self, vocab, k=20):
        """Most negative / positive weighted words: inspect these. If 'profit' is negative, something is wrong."""
        inv = {i: w for w, i in vocab.items()}
        w = self.dense.W[:, 2] - self.dense.W[:, 0]                   # positive minus negative weight
        order = np.argsort(w)
        return [inv[i] for i in order[:k]], [inv[i] for i in order[-k:]]
```

src/sentiment/naive_bayes.py

```
import numpy as np


class NaiveBayes:
    """Multinomial Naive Bayes on raw count vectors with add-one (Laplace) smoothing."""
    def fit(self, Xc, y, n_classes=3):
        self.log_prior = np.log(np.bincount(y, minlength=n_classes) / len(y))
        self.log_lik = np.zeros((n_classes, Xc.shape[1]))
        for c in range(n_classes):
            counts = Xc[y == c].sum(axis=0) + 1.0
            self.log_lik[c] = np.log(counts / counts.sum())
        return self

    def predict(self, Xc):
        return np.argmax(Xc @ self.log_lik.T + self.log_prior, axis=1)
```

src/sentiment/daily_signal.py

```
"""Headlines -> three daily numbers, aligned to trading days without look-ahead."""
import numpy as np
import pandas as pd


def map_to_trading_day(dates: pd.Series, trading_days: pd.DatetimeIndex) -> pd.Series:
    """Each headline date -> first trading day ON OR AFTER it (weekend/holiday news rolls forward, because
    that is when the market reacts). Day t therefore only receives headlines dated t or earlier."""
    td = np.sort(trading_days.values)
    idx = np.searchsorted(td, dates.values, side="left")
    out = pd.Series(pd.NaT, index=dates.index, dtype="datetime64[ns]")
    ok = idx < len(td)
    out[ok] = td[idx[ok]]
    return out


def score_headlines(texts, tokenizer, tfidf, model, chunk=10_000):
    """Chunked scoring (dense TF-IDF over millions of rows would exhaust memory). Returns (n, 3) probs."""
    probs = []
    for s in range(0, len(texts), chunk):
        X = tfidf.transform([tokenizer(t) for t in texts[s:s + chunk]])
        probs.append(model.predict_proba(X))
    return np.vstack(probs)


def aggregate_daily(mapped_days: pd.Series, probs: np.ndarray) -> pd.DataFrame:
    df = pd.DataFrame({
        "date": mapped_days.values,
        "score": probs[:, 2] - probs[:, 0],                          # P(positive) - P(negative)
        "is_neg": (probs.argmax(axis=1) == 0).astype(float),
    }).dropna(subset=["date"])
    g = df.groupby("date")
    out = pd.DataFrame({"sent_mean": g["score"].mean(), "sent_neg_share": g["is_neg"].mean(),
                        "sent_log_count": np.log(g.size())})
    return out    # days without headlines are filled with neutral values when merged: 0, 0, 0
```

7.9 Evaluation (Phase 7)

All models are scored by the same code from their forecast CSVs. **QLIKE** is computed stably as `exp(d) - d - 1` with `d = log RV_actual - log forecast`, which equals `RV/h - log(RV/h) - 1`; it is 0 for a perfect forecast and punishes under-prediction more. The **Diebold-Mariano** test compares loss differences with a Newey-West (Bartlett) variance over about 5 lags. The **VaR backtest** sets `VaR = 2.33 x exp(forecast/2)` (volatility, not variance), counts days where the actual open-to-close loss exceeded it, and applies the **Kupiec** likelihood-ratio test against a 1% breach rate (reject if LR > 3.84).

src/eval/metrics.py

```
import numpy as np
import pandas as pd
from src.config import TEST_START, TEST_END, VAL_START, VAL_END
from src.io import read_features, read_forecast

PERIODS = {"val": (VAL_START, VAL_END), "test": (TEST_START, TEST_END)}


def mse_log(y, f):
    return float(np.mean((np.asarray(y) - np.asarray(f)) ** 2))


def qlike_losses(y_log_rv, f_log_rv):
    """Per-day QLIKE = RV/h - log(RV/h) - 1, with RV/h = exp(y - f). Exactly 0 when forecast == actual."""
    d = np.asarray(y_log_rv, float) - np.asarray(f_log_rv, float)
    return np.exp(d) - d - 1.0


def load_aligned(name, period="test") -> pd.DataFrame:
    """Join a forecast CSV with the realised target and return for the chosen period (by target_date)."""
    lo, hi = PERIODS[period]
    df = read_features()[["date", "target_date", "target_log_rv", "target_oc_ret"]]
    m = df.merge(read_forecast(name), on=["date", "target_date"], how="inner")
    return m[(m["target_date"] >= lo) & (m["target_date"] <= hi)].reset_index(drop=True)
```

src/eval/dm_test.py

```
import math
import numpy as np


def dm_test(loss_a, loss_b, lags=5):
    """H0: equal expected loss. d_t = loss_a - loss_b. DM < 0 (significant) means model A is better.
    Newey-West variance: gamma_0 + 2 * sum_k (1 - k/(L+1)) * gamma_k, divided by n. Returns (DM, p_value)."""
    d = np.asarray(loss_a, float) - np.asarray(loss_b, float)
    n, dbar = len(d), d.mean()
    var = np.mean((d - dbar) ** 2)
    for k in range(1, lags + 1):
        var += 2.0 * (1.0 - k / (lags + 1)) * np.mean((d[k:] - dbar) * (d[:-k] - dbar))
    var /= n
    if var <= 0:
        return 0.0, 1.0                                          # identical series
    dm = dbar / math.sqrt(var)
    return float(dm), float(math.erfc(abs(dm) / math.sqrt(2)))      # two-sided normal p-value
```

src/eval/var_backtest.py

```
import math
import numpy as np
from src.config import VAR_Z, VAR_LEVEL
from src.io import log_rv_to_vol


def var_breaches(forecast_log_rv, oc_ret_next, z=VAR_Z):
    """Breach = actual open-to-close LOSS exceeds z x forecast daily volatility."""
    var = z * log_rv_to_vol(forecast_log_rv)
    return (-np.asarray(oc_ret_next)) > var


def kupiec(n, x, p=VAR_LEVEL):
    """Proportion-of-failures LR test. Returns (LR, p_value); reject H0 (correct 1% rate) if LR > 3.84."""
    def loglik(q):
        return (n - x) * math.log1p(-q) + (x * math.log(q) if x > 0 else 0.0)
    lr = -2.0 * (loglik(p) - loglik(x / n))
    return float(lr), float(math.erfc(math.sqrt(max(lr, 0.0) / 2.0)))   # chi-square(1) survival function
```

src/eval/walk_forward.py

```
import pandas as pd


def walk_forward(df, fit_predict, start="2021-01-01", end="2023-06-30"):
    """Refit at the start of each quarter on rows whose TARGET lies before that quarter, forecast the quarter.
    fit_predict(train_df, test_df) -> np.ndarray of forecast_log_rv for test_df rows."""
    parts = []
    for qs in pd.date_range(start, end, freq="QS"):
        qe = min(qs + pd.offsets.QuarterEnd(0), pd.Timestamp(end))
        train, test = df[df["target_date"] < qs], df[(df["target_date"] >= qs) & (df["target_date"] <= qe)]
        if len(test):
            parts.append(test[["date", "target_date"]].assign(forecast_log_rv=fit_predict(train, test)))
    return pd.concat(parts, ignore_index=True)
```

Walk-forward cost: refitting HAR and GARCH each quarter is cheap; neural models retrain every quarter with 5 seeds (cut to 3 seeds if needed and say so). Time the GARCH loop before putting it inside the walk-forward.

7.10 Entry-point scripts

Scripts only wire modules together; no logic that results depend on lives in notebooks. Run each with `python -m scripts.<name>` from the repository root.

scripts/build_features.py

```
"""Phase 1: raw files -> data/processed/daily_features.csv (+ dropped-days log)."""
import pandas as pd
from src.config import DATA_RAW, DATA_PROC, DATA_START, DATA_END
from src.data.load_minute import load_minute, to_5min
from src.data.load_aux import load_vix, load_turnover, load_expiry_dates
from src.data.realized_vol import daily_from_bars
from src.data.features import build_features


def main():
    minute = load_minute(DATA_RAW / "nifty_1min.csv")               # TODO file name; check first bar = 09:15
    daily, dropped = daily_from_bars(to_5min(minute))
    daily = daily.loc[DATA_START:DATA_END]
    feats = build_features(daily, load_vix(DATA_RAW / "india_vix.csv"),
                           load_turnover(DATA_RAW / "nifty_turnover.csv"),
                           load_expiry_dates(DATA_RAW / "expiry_dates.csv"))
    DATA_PROC.mkdir(parents=True, exist_ok=True)
    feats.to_csv(DATA_PROC / "daily_features.csv", index=False)
    pd.DataFrame(dropped, columns=["date", "n_bars"]).to_csv(DATA_PROC / "dropped_days.csv", index=False)
    lo, hi = feats["date"].min().date(), feats["date"].max().date()
    print(f"rows={len(feats)} range={lo}..{hi} dropped={len(dropped)}")
    corr = feats["log_rv"].corr(feats["target_log_rv"])
    print("corr(log_rv_t, log_rv_t+1) =", round(corr, 3), "(should be clearly positive)")


if __name__ == "__main__":
    main()
```

scripts/run_baselines.py

```
"""Phase 2: Naive, HAR, HAR-X, GARCH forecasts for validation + test rows. Prints VALIDATION numbers only."""
import numpy as np
from src.config import TARGET, HAR_X_EXTRA, SENT_FEATURES
from src.data.split import split_masks
from src.eval.metrics import mse_log, qlike_losses
from src.io import read_features, write_forecast
from src.models.garch import fit_garch, garch_log_rv_forecast, unpack
from src.models.har import har_design, fit_har, predict_har
from src.models.naive import naive_forecast


def main():
    df = read_features()
    m = split_masks(df)
    y, keep = df[TARGET].to_numpy(), m["val"] | m["test"]
    out = {"naive": naive_forecast(df)}

    for name, extra in [("har", ()), ("har_x", HAR_X_EXTRA), ("har_x_sent", HAR_X_EXTRA + SENT_FEATURES)]:
        if any(c in extra for c in SENT_FEATURES) and df[SENT_FEATURES].isna().any().any():
            continue                                                # sentiment not built yet (Phase 5)
        beta = fit_har(har_design(df[m["train"]], extra), y[m["train"]])
        out[name] = predict_har(har_design(df, extra), beta)
        print(name, "coefficients:", np.round(beta, 3))

    r = df["oc_ret"].to_numpy() * 100.0                             # percent, open-to-close
    theta = fit_garch(r[m["train"]])
    out["garch"] = garch_log_rv_forecast(theta, r, s2_start=r[m["train"]].var())
    om, al, be = unpack(theta)
    print("GARCH omega/alpha/beta/persistence:", round(om, 5), round(al, 4), round(be, 4), round(al + be, 4))

    for name, f in out.items():
        write_forecast(name, df[keep], f[keep])
        v = m["val"]
        print(f"{name:11s} val MSE={mse_log(y[v], f[v]):.4f}  val QLIKE={qlike_losses(y[v], f[v]).mean():.4f}")


if __name__ == "__main__":
    main()
```

scripts/train_nn.py

```
"""Phases 3, 4, 6: train MLP / LSTM / Transformer (with or without sentiment) over 5 seeds."""
import argparse
from src.config import (BASE_FEATURES, SENT_FEATURES, WINDOW_MLP, WINDOW_SEQ, MLP_CFG, LSTM_CFG, TRF_CFG)
from src.models.common import run_model
from src.models.mlp import build_mlp
from src.models.lstm import build_lstm
from src.models.transformer import build_transformer

SPECS = {   # name -> (features, window, build function, training config)
    "mlp": (BASE_FEATURES, WINDOW_MLP, build_mlp, MLP_CFG),
    "lstm": (BASE_FEATURES, WINDOW_SEQ, build_lstm, LSTM_CFG),
    "transformer_nosent": (BASE_FEATURES, WINDOW_SEQ, build_transformer, TRF_CFG),
    "transformer_sent": (BASE_FEATURES + SENT_FEATURES, WINDOW_SEQ, build_transformer, TRF_CFG),
}

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--model", required=True, choices=SPECS)
    name = ap.parse_args().model
    features, window, build, cfg = SPECS[name]
    run_model(name, features, window, build, cfg)
```

scripts/run_all_eval.py

```
"""Phase 7: every number in the report comes from this one script (run ONLY after settings are frozen)."""
import pandas as pd
from src.config import MODEL_NAMES, REPORT, VAR_LEVEL
from src.eval.metrics import load_aligned, mse_log, qlike_losses
from src.eval.dm_test import dm_test
from src.eval.var_backtest import var_breaches, kupiec


def main(period="test", benchmark="har"):
    REPORT.mkdir(exist_ok=True)
    main_rows, dm_rows, var_rows = [], [], []
    ref = qlike_losses(*load_aligned(benchmark, period)[["target_log_rv", "forecast_log_rv"]].T.to_numpy())
    for name in MODEL_NAMES:
        try:
            a = load_aligned(name, period)
        except FileNotFoundError:
            continue
        assert len(a) == len(ref), f"{name}: rows differ from benchmark"   # DM needs identical days
        y, f = a["target_log_rv"].to_numpy(), a["forecast_log_rv"].to_numpy()
        q = qlike_losses(y, f)
        main_rows.append({"model": name, "n": len(a), "MSE_logRV": mse_log(y, f), "QLIKE": q.mean()})
        dm, p = dm_test(q, ref)                                  # negative DM: better than the benchmark
        dm_rows.append({"model": name, "vs": benchmark, "DM": dm, "p_value": p})
        n, x = len(a), int(var_breaches(f, a["target_oc_ret"].to_numpy()).sum())
        lr, pk = kupiec(n, x)
        var_rows.append({"model": name, "breaches": x, "expected": VAR_LEVEL * n,
                         "kupiec_LR": lr, "result": "FAIL" if lr > 3.84 else "pass"})
    for fname, rows in [("results_main", main_rows), ("results_dm", dm_rows), ("results_var", var_rows)]:
        pd.DataFrame(rows).to_csv(REPORT / f"{fname}_{period}.csv", index=False)
        print(pd.DataFrame(rows).round(4).to_string(index=False), "\n")


if __name__ == "__main__":
    main()
```

`scripts/build_sentiment.py` (not shown in full) follows section 7.8: split PhraseBank 80/20 stratified with a fixed seed, fit `Tfidf` on the train split, train `SoftmaxRegression` and `NaiveBayes`, print accuracy and macro F1 on the 20% split **and** on the 400 Economic Times items, filter the Times of India archive to business categories (inspect `headline_category` values per year first), `score_headlines` in chunks, `map_to_trading_day`, `aggregate_daily`, write `sentiment_daily.csv`, fill missing days with 0, and merge the three columns into `daily_features.csv`.

8. GitHub Copilot guidance

8.1 .github/copilot-instructions.md

Paste this file into the repository. Copilot includes it as context for every chat and completion, so the project rules are always applied.

.github/copilot-instructions.md

```
# Project: Nifty Volatility Forecaster (NumPy-from-scratch)

Forecast next-day Nifty 50 realized volatility (target = log realized VARIANCE of day t+1). Compare Naive,
HAR-RV, GARCH(1,1), MLP, LSTM and a time-series Transformer. Full spec: docs/PROJECT_BLUEPRINT.md.

## Hard rules
- Allowed imports: numpy, pandas, matplotlib and the Python standard library ONLY.
- NEVER import or suggest scikit-learn, torch, tensorflow, keras, jax, statsmodels, arch, nltk, spacy, scipy,
  transformers, xgboost, lightgbm, gensim. NEVER load pretrained models, embeddings or checkpoints.
- Every neural layer is hand-written with forward(x, training), backward(dout), own_params() returning
  (value, grad) array pairs; gradients ACCUMULATE (+=) after zero_grad(). Use float64.
- Every new layer needs a gradient check in tests/test_gradcheck.py (relative error < 1e-6).
- Seeds: pass np.random.default_rng(seed) explicitly. No global random state.

## Data rules (avoid leakage)
- Split by target_date: train <= 2019-12-31, val = 2020, test = 2021-01-01..2023-06-30.
- Scaler and target statistics are fitted on TRAIN rows only. No backward fill. Rolling features look back only.
- Never use the test period for tuning. The test set is opened once, in scripts/run_all_eval.py.
- Units: log_rv is log of variance. Volatility = exp(log_rv/2) via src.io.log_rv_to_vol.

## Style
- Put the array shape in a comment next to every non-trivial array: # (batch, time, features).
- Small pure functions, type hints on public functions, docstring explaining WHY, not just what.
- Models write forecasts to forecasts/<name>.csv with columns date, target_date, forecast_log_rv.
- src/eval must not import src/models; it reads forecast CSVs only.
```

8.2 Per-module prompts for Copilot Chat

Attach `docs/PROJECT_BLUEPRINT.md` and the relevant skeleton file, then use one prompt per module. Each prompt names the file, the contract and the acceptance test, so Copilot has a concrete target.

Step	Prompt (shortened)
1	Using #file:docs/PROJECT_BLUEPRINT.md section 7.2, implement TODOs in src/data/load_minute.py and load_aux.py for the real Kaggle/NSE column names in data/raw/. Keep the function signatures. Add a test.
2	Implement scripts/build_features.py end to end and print row count, date range, dropped days and the correlation of log_rv with target_log_rv. Do not change src/data/features.py without updating tests.
3	Run and fix tests/test_gradcheck.py. If a check fails, derive the backward pass on paper in a comment first, then correct it. Do not loosen the 1e-6 tolerance.
4	Write scripts/build_sentiment.py following section 7.8. Use only src/sentiment and src/nn modules. Report PhraseBank and Economic Times accuracy and macro F1.
5	Add a notebook-free script that plots actual vs HAR vs LSTM log RV over 2020 with matplotlib to report/fig_2020_forecasts.png, using forecasts/*.csv only.
6	Add src/eval/attention_plot.py that averages TimeSeriesTransformer.mean_attention over the test windows and saves report/fig_attention.png with a one-line interpretation printed to the console.

9. Tests, CI and phase checklist

9.1 Tests (they define 'done' for the skeleton)

tests/test_gradcheck.py

```
import numpy as np
from src.nn.gradcheck import gradcheck_layer, gradcheck_loss
from src.nn.layers import Dense, ReLU, GELU, LayerNorm, Softmax, Sequential
from src.nn.losses import MSELoss, SoftmaxCrossEntropy
from src.models.lstm import LSTM
from src.models.transformer import MultiHeadAttention, EncoderBlock, TimeSeriesTransformer

TOL = 1e-6


def _ok(errs):
    assert max(errs.values()) < TOL, errs


def test_dense_relu_gelu_layernorm_softmax():
    rng = np.random.default_rng(0)
    _ok(gradcheck_layer(Dense(5, 4, rng), rng.standard_normal((3, 5)), rng))
    _ok(gradcheck_layer(Dense(5, 4, rng), rng.standard_normal((2, 3, 5)), rng))     # 3-D input
    _ok(gradcheck_layer(ReLU(), rng.standard_normal((3, 5)) + 0.1, rng))
    _ok(gradcheck_layer(GELU(), rng.standard_normal((3, 5)), rng))
    ln = LayerNorm(6); ln.gamma[:] = rng.standard_normal(6); ln.beta[:] = rng.standard_normal(6)
    _ok(gradcheck_layer(ln, rng.standard_normal((2, 4, 6)), rng))
    _ok(gradcheck_layer(Softmax(), rng.standard_normal((3, 5)), rng))


def test_losses():
    rng = np.random.default_rng(1)
    assert gradcheck_loss(MSELoss(), rng.standard_normal((4, 1)), rng.standard_normal(4)) < TOL
    assert gradcheck_loss(SoftmaxCrossEntropy(), rng.standard_normal((5, 3)), np.array([0, 1, 2, 1, 0])) < TOL


def test_lstm_tiny_case():                                          # 2 inputs, 3 hidden, 4 steps, batch 2
    rng = np.random.default_rng(2)
    _ok(gradcheck_layer(LSTM(2, 3, rng), rng.standard_normal((2, 4, 2)), rng))


def test_attention_block_and_full_transformer():
    rng = np.random.default_rng(3)
    _ok(gradcheck_layer(MultiHeadAttention(8, 2, rng), rng.standard_normal((2, 4, 8)), rng))
    _ok(gradcheck_layer(EncoderBlock(8, 2, rng, p=0.0), rng.standard_normal((2, 4, 8)), rng))
    _ok(gradcheck_layer(TimeSeriesTransformer(3, 4, rng, d=8, heads=2, p=0.0), rng.standard_normal((2, 4, 3)), rng))
```

tests/test_metrics.py

```
import numpy as np
from src.eval.metrics import qlike_losses
from src.eval.dm_test import dm_test
from src.eval.var_backtest import kupiec


def test_qlike_zero_for_perfect_forecast():
    y = np.log(np.array([1e-4, 3e-4, 2e-5]))
    assert np.allclose(qlike_losses(y, y), 0.0)


def test_qlike_punishes_underprediction_more():
    y = np.array([0.0])
    assert qlike_losses(y, y - 1.0)[0] > qlike_losses(y, y + 1.0)[0]


def test_dm_identical_series_is_zero_and_noisy_copy_is_significant():
    rng = np.random.default_rng(0)
    y = rng.standard_normal(600)
    good = y + 0.1 * rng.standard_normal(600)
    bad = y + 0.8 * rng.standard_normal(600)
    assert dm_test(qlike_losses(y, good), qlike_losses(y, good))[0] == 0.0
    dm, p = dm_test(qlike_losses(y, good), qlike_losses(y, bad))
    assert dm < -1.96 and p < 0.05


def test_kupiec_accepts_expected_breaches_and_rejects_many():
    assert kupiec(620, 6)[0] < 3.84
    assert kupiec(620, 25)[0] > 3.84
```

tests/test_no_leakage.py

```
import numpy as np
import pandas as pd
from src.config import BASE_FEATURES
from src.data.features import build_features


def _synthetic(rng, n=120):
    idx = pd.bdate_range("2020-01-01", periods=n)
    close = 100 * np.exp(np.cumsum(rng.normal(0, 0.01, n)))
    daily = pd.DataFrame({"open": close * (1 + rng.normal(0, 0.002, n)), "high": close * 1.01,
                          "low": close * 0.99, "close": close, "rv": rng.uniform(1e-5, 1e-4, n),
                          "oc_ret": rng.normal(0, 0.01, n)}, index=idx)
    vix = pd.DataFrame({"vix_close": rng.uniform(12, 25, n)}, index=idx)
    tov = pd.DataFrame({"turnover": rng.uniform(1e4, 2e4, n)}, index=idx)
    return daily, vix, tov


def test_features_do_not_use_future_rows():
    rng = np.random.default_rng(0)
    daily, vix, tov = _synthetic(rng)
    expiry = set(daily.index[::5])
    base = build_features(daily, vix, tov, expiry)
    cut = daily.index[80]
    d2, v2, t2 = daily.copy(), vix.copy(), tov.copy()
    for frame in (d2, v2, t2):                                      # corrupt everything AFTER the cutoff
        frame.loc[frame.index > cut] *= 3.0
    changed = build_features(d2, v2, t2, expiry)
    a, b = base[base["date"] <= cut], changed[changed["date"] <= cut]
    pd.testing.assert_frame_equal(a[BASE_FEATURES].reset_index(drop=True), b[BASE_FEATURES].reset_index(drop=True))
```

tests/test_no_banned_imports.py

```
"""Enforces the project rule: no ML libraries and no pretrained models anywhere in the code base."""
import ast
from pathlib import Path

BANNED = {"sklearn", "torch", "tensorflow", "keras", "statsmodels", "arch", "nltk", "spacy", "scipy",
          "transformers", "jax", "xgboost", "lightgbm", "gensim", "onnx", "onnxruntime"}
ROOT = Path(__file__).resolve().parents[1]


def test_no_banned_imports_or_pretrained_words():
    for folder in ("src", "scripts", "tests"):
        for path in (ROOT / folder).rglob("*.py"):
            if path.name == Path(__file__).name:
                continue
            src = path.read_text(encoding="utf-8")
            assert "from_pretrained" not in src and "pretrained" not in src.lower(), f"pretrained in {path}"
            for node in ast.walk(ast.parse(src)):
                mods = []
                if isinstance(node, ast.Import):
                    mods = [a.name.split(".")[0] for a in node.names]
                elif isinstance(node, ast.ImportFrom) and node.module:
                    mods = [node.module.split(".")[0]]
                assert not (set(mods) & BANNED), f"banned import {mods} in {path}"
```

tests/run_tests.py

```
"""Minimal runner so tests work without pytest:  python -m tests.run_tests"""
import importlib
import sys
import time
import traceback

MODULES = ["tests.test_gradcheck", "tests.test_metrics", "tests.test_no_leakage", "tests.test_no_banned_imports"]

if __name__ == "__main__":
    failed = 0
    for mod_name in MODULES:
        mod = importlib.import_module(mod_name)
        for name in sorted(n for n in dir(mod) if n.startswith("test_")):
            t0 = time.time()
            try:
                getattr(mod, name)()
                print(f"PASS {mod_name}.{name} ({time.time() - t0:.1f}s)")
            except Exception:
                failed += 1
                print(f"FAIL {mod_name}.{name}")
                traceback.print_exc()
    sys.exit(1 if failed else 0)
```

.github/workflows/tests.yml

```
name: tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: python -m tests.run_tests
```

9.2 Phase checklist (what each phase must produce)

Phase / weeks	Deliverable	Gate / done when
0 Setup (2 days)	repo, requirements, README rule, SOURCES.md, professor approval of pandas + np.linalg	datasets downloaded
1 Data (wk 1-2)	daily_features.csv, split.py, fig_rv_history.png	corr(logRV_t, logRV_t+1) clearly positive; leakage test passes; GATE 1: no neural training before this file exists
2 Baselines (wk 3)	naive, har, har_x, garch forecasts + validation table	HAR beats Naive on val QLIKE; HAR coeffs positive, sum a little < 1; GARCH persistence 0.9-0.995
3 NN core + MLP (wk 3-4)	layers, optim, gradcheck, trainer, mlp.csv	all gradient checks < 1e-6; MLP overfits 64 rows
4 LSTM (wk 5)	LSTM layer, lstm.csv (5 seeds)	cell + layer gradient checks; can overfit 64 sequences
5 Sentiment (wk 2-5, parallel)	sentiment_daily.csv merged into features	LogReg >= NB on PhraseBank macro F1; Indian-news accuracy reported; sensible top words
6 Transformer (wk 6-7)	attention parts, two Transformer variants, attention heatmap	every component gradient-checked; stable over 5 seeds; GATE 2: freeze settings, tag 'pre-test' (end wk 7)
7 Evaluation (wk 8-9)	results, DM, VaR/Kupiec, ablations, walk-forward plot	one script (run_all_eval.py) regenerates every number
8 Demo + report (wk 9-10)	static demo (3 views), 12-15 page report, 10-minute talk	demo runs with no install step; report numbers match script output

9.3 Risks and fallbacks (condensed from the roadmap)

- **Shorter deadline (~6 weeks):** drop LSTM and walk-forward, keep MLP and Transformer, fixed split only.
- **Transformer backward will not pass gradient checks by mid week 7:** 1 head, no dropout; if still stuck, present the LSTM as top model and the Transformer forward pass with attention plots as exploratory.
- **Minute data gaps (> 5% of days dropped):** use Parkinson range-based volatility for affected years and say so.
- **Weak sentiment classifier on Indian news:** report it, keep the feature, add a hand-made negative-word lexicon score as a second feature.
- **Neural models lose to HAR:** not a failure; it matches published research. Report with the DM test.
- **Professor rejects pandas:** load CSVs with the `csv` module into NumPy arrays (about a day of work).
- If time runs short, protect the core: data, HAR, GARCH, one neural model, the sentiment ablation, the DM test.

9.4 Pitfalls checklist for code review

- Timezone: first bar of each day must be 09:15 IST (UTC data: 03:45).
- exp(log RV) is variance; volatility is its square root. Always use `log_rv_to_vol`.
- Scale GARCH returns to percent, convert the forecast back (divide variance by 1e4).
- Every layer must be gradient-checked in float64 before training; shapes in comments.
- Shared LSTM weights: gradients are summed over steps, never overwritten.

- Overlapping 22-day windows are highly correlated: do not count them as independent samples.
- Never tune on the test set; if the Transformer cannot beat HAR on validation, report it as is.
- Process headlines in chunks of 10,000; check headline coverage per year (category names change).

10. Initial commit procedure

10.1 Commands

terminal

```
git clone https://github.com/<you>/nifty-vol.git && cd nifty-vol
mkdir -p docs data/raw data/processed forecasts/seeds notebooks report .github/workflows
mkdir -p src/data src/nn src/models src/sentiment src/eval scripts tests
for d in src src/data src/nn src/models src/sentiment src/eval scripts tests; do touch $d/__init__.py; done
touch data/raw/.gitkeep data/processed/.gitkeep forecasts/.gitkeep report/.gitkeep

# paste the files from sections 4, 7, 8, 9 here, copy the blueprint into docs/
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python -m tests.run_tests                              # gradient checks + metrics + banned-import scan

git add -A
git commit -m "chore: initial project skeleton (NumPy-from-scratch volatility forecaster)"
git push -u origin main
```

Tag milestones as you go: `git tag phase1-data`, ... and `git tag pre-test` at the end of week 7 once all settings are frozen (this proves nothing was tuned on test data).

10.2 data/raw/SOURCES.md template

data/raw/SOURCES.md

```
# Data sources (record URL and download date for every file)

| File | Source URL | Downloaded | Notes |
|------|-----------|------------|-------|
| nifty_1min.csv | <Kaggle dataset URL> | YYYY-MM-DD | 1-minute OHLC, no volume; check timezone |
| india_vix.csv | <NSE VIX history URL> | YYYY-MM-DD | daily OHLC of India VIX |
| nifty_turnover.csv | <NSE daily data URL> | YYYY-MM-DD | shares traded, turnover (Rs crore) |
| expiry_dates.csv | <NSE circulars> | YYYY-MM-DD | actual expiry dates; weekday changed over time |
| india_news_headlines.csv | <Kaggle TOI archive URL> | YYYY-MM-DD | 2001 to Jun 2023, ~3.9M rows |
| financial_phrasebank.csv | <Malo et al. 2014 source> | YYYY-MM-DD | use >=66% agreement version |
| economic_times_labelled.csv | <Kaggle URL> | YYYY-MM-DD | 400 labelled headlines |
```

10.3 README.md skeleton

README.md

```
# Nifty Volatility Forecaster

Forecasts next-day Nifty 50 realized volatility with six models, from a naive rule to a Transformer written
from scratch in NumPy. No ML libraries and no pretrained models (enforced by tests/test_no_banned_imports.py).

## Results (fill after Phase 7)
| Model | Test MSE (log RV) | Test QLIKE | DM vs HAR (p) | VaR breaches (expected) | Kupiec |
|-------|-------------------|------------|---------------|-------------------------|--------|

## Data
Row count, date range and dropped days: <fill after Phase 1>. Sources: data/raw/SOURCES.md.

## Reproduce
    pip install -r requirements.txt
    make features sentiment baselines nn
    make eval          # regenerates every number in the report
    make test          # gradient checks, leakage test, banned-import scan

## Rules
Allowed: Python, NumPy, pandas (CSV loading), matplotlib. Not allowed: scikit-learn, PyTorch, TensorFlow,
statsmodels, arch, NLTK, spaCy, scipy, HuggingFace, or any pretrained weights.
```

10.4 Suggested first-week order of work

- Commit this skeleton and get `python -m tests.run_tests` green (it only needs NumPy and pandas, no data).
- Email the professor for written approval of pandas and `np.linalg` (Phase 0).
- Download datasets, fill `SOURCES.md`, then implement the TODOs in `load_minute.py` / `load_aux.py`.

- Produce `daily_features.csv`; check the first bar is 09:15 IST, the RV history plot shows March 2020, and the day-to-day log RV correlation is strongly positive. This unlocks everything else (Gate 1).